

Examen final de Programació i Estructures de Dades (PRD)

Data: 30 de maig de 2025. Duració: 2h 30min.

Publicació de notes provisionals: 06/06/2025 (Atenea)

Revisió de l'examen: 10/06/2025 – 11:30h, Sala D6-221 (edifici D6, 2ª planta)

IMPORTANT: cal resoldre i lliurar els problemes 1, 2 i 3 en fulles separades

PROBLEMA 1: 4 punts, temps de resolució: 50 min.

Volem implementar una cua de paquets de dades, similar a les que vam implementar a la primera part del projecte de laboratori d'aquest quadrimestre (simulador d'un sistema de cues de paquets). En aquest cas però, disposem d'una cua única, capaç d'emmagatzemar i extreure paquets de qualsevol flux de dades seguint una disciplina FIFO (*First In, First Out*). Per tal d'aconseguir-ho, hem definit les constants simbòliques i estructures de dades següents:

```
#define MAX_P      500          /* Capacitat màxima de la cua      */
#define PRIORITZA  0           /* Acció de prioritzar paquet     */
#define PENALITZA  1           /* Acció de penalitzar paquet     */

typedef struct
{
    int id_flux;                /* Identificador de flux          */
    int id_paquet;              /* Identificador del paquet       */
    int mida_bytes;            /* Mida en bytes del paquet      */
}t_paquet;

typedef struct
{
    int npaquets;                /* Nombre de paquets emmagatzemats */
    t_paquet paquets[MAX_P];    /* Paquets emmagatzemats          */
}t_cua;
```

A partir d'aquesta descripció inicial, cal implementar les funcions següents:

- a. **(0.5 punts)** `int afegeix_nou_paquet (t_cua *cua, t_paquet p);` aquesta funció ha d'afegir el nou paquet `p` passat com a paràmetre al final de la cua de paquets apuntada pel paràmetre `cua`. En cas que la cua de paquets estigui completament plena i per tant el nou paquet no pugui afegir-se en aquesta, la funció ha d'acabar retornant directament el valor `-1`. Altrament, si el nou paquet pot afegir-s'hi de forma satisfactòria, la funció ha d'acabar retornant `0`.

```
int afegeix_nou_paquet (t_cua *cua, t_paquet p)
{
    if (cua->npaquets == MAX_P)
        return (-1);

    cua->paquets[cua->npaquets] = p;
    cua->npaquets++;
    return (0);
}
```

- b. **(1 punt)** `int max_num_paquets_consecutius (t_cua *cua, int id_f);` aquesta funció ha de trobar i retornar el nombre de paquets consecutius màxim amb identificador de flux igual al paràmetre `id_f` desats a la cua de paquets apuntada pel paràmetre `cua`.

Per exemple, imaginem que la cua de paquets apuntada pel paràmetre `cua` guarda actualment 10 paquets de dades amb identificadors de flux iguals a:

{1, 1, 2, 2, 2, 1, 1, 1, 3, 3}

En cas que el valor del paràmetre `id_f` sigui igual a 1, la funció hauria d'acabar retornant 3 (existeix un màxim de 3 paquets consecutius amb identificador de flux igual a 1 a les posicions 5, 6 i 7 de la cua). Altrament, en cas que el valor d'aquest paràmetre sigui igual a 3, la funció hauria d'acabar retornant 2 (existeixen 2 paquets amb identificador de flux igual a 3 a les posicions 8 i 9 de la cua).

```
int max_num_paquets_consecutius (t_cua *cua, int id_f)
{
    int i, cons = 0, max_cons = 0;

    for (i = 0; i < cua->npaquets; i++)
    {
        if (cua->paquets[i].id_flux == id_f)
        {
            cons++;

            if (max_cons < cons)
                max_cons = cons;
        }
        else
            cons = 0;
    }

    return (max_cons);
}
```

- c. **(1 punt)** `int elimina_tots_flux (t_cua *cua, int id_f);` aquesta funció ha d'eliminar tots els paquets amb identificador de flux igual al paràmetre `id_f` guardats a la cua de paquets apuntada pel paràmetre `cua`. A més, la funció ha d'acabar retornant el nombre total de paquets eliminats durant l'operació.

```
int elimina_tots_flux (t_cua *cua, int id_f)
{
    int i, j, elim = 0;

    for (i = 0; i < cua->npaquets; i++)
    {
        if (cua->paquets[i].id_flux == id_f)
        {
            for (j = i+1; j < cua->npaquets; j++)
                cua->paquets[j-1] = cua->paquets[j];

            elim++;
            cua->npaquets--;
            i--;
        }
    }
}
```

```

    }

    return (elim);
}

```

- d. **(1.5 punts)** `int canvia_posicio_paquet (t_cua *cua, int id_f, int id_p, int accio);` aquesta funció rep com a paràmetres un punter a una cua de paquets (`cua`), un identificador de flux (`id_f`), un identificador de paquet (`id_p`) i una acció a realitzar (`accio`). Aquesta acció a realitzar pot ser tan sols igual a les constants simbòliques `PRIORITZA` o `PENALITZA`, definides al programa.

En primer lloc, la funció ha de cercar a la cua el paquet amb identificador de flux i de paquet iguals als valors dels paràmetres `id_f` i `id_p`. En cas de no trobar-lo a la cua de paquets, o si el valor del paràmetre `accio` no coincideix amb cap d'aquestes dues constants definides al programa, la funció ha de retornar directament -1.

En cas de trobar el paquet buscat, quan el valor del paràmetre `accio` és igual a la constant `PRIORITZA`, la funció ha de moure aquest paquet a la primera posició (posició 0) de la cua. Altrament, quan el valor del paràmetre `accio` és igual a la constant `PENALITZA`, la funció ha de moure el paquet al final de la cua (primera posició lliure d'aquesta). En qualsevol d'aquests dos casos, el paquet ha de quedar eliminat de la posició que ocupava anteriorment (posició on s'havia trobat el paquet inicialment). Finalment, la funció ha d'acabar retornant 0 per tal d'indicar que la operació ha estat satisfactòria.

```

int canvia_posicio_paquet (t_cua *cua, int id_f, int id_p, int accio)
{
    int i, pos = -1;
    t_paquet paquet;

    for (i = 0; i < cua->npaquets && pos == -1; i++)
    {
        if (cua->paquets[i].id_flux == id_f &&
            cua->paquets[i].id_paquet == id_p)
        {
            pos = i;
            paquet = cua->paquets[i];
        }
    }

    if (pos == -1 || (accio != PRIORITZA && accio != PENALITZA))
        return (-1);

    if (accio == PRIORITZA)
    {
        for (i = pos; i > 0; i--)
            cua->paquets[i] = cua->paquets[i-1];

        cua->paquets[0] = paquet;
    }
    else
    {
        for (i = pos+1; i < cua->npaquets; i++)
            cua->paquets[i-1] = cua->paquets[i];
    }
}

```

```

        cua->paquets[cua->npaquets-1] = paquet;
    }

    return (0);
}

```

PROBLEMA 2: 4.5 punts, temps de resolució: 75 min.

En aquest problema es sol·licita la implementació de la funcionalitat associada a una estructura de dades capaç d'emmagatzemar dins de llistes enllaçades les mesures que un conjunt de sensors d'un sistema IoT (*Internet of Things*) poden prendre al llarg del temps. Amb aquest objectiu, el disseny del programa contempla la definició de les constants simbòliques i estructures de dades següents:

```

#define NUM_S 10                                /* Nombre de sensors del sistema IoT */

typedef struct node {
    int hora;                                   /* Hora de la mesura (format: hhmm) */
    float valor;                               /* Valor mesurat */
    struct node *next;                         /* Punter a següent node de la llista */
}t_node;

typedef struct {
    t_node *sensors[NUM_S];                  /* Array de sensors del sistema */
}t_sistema;

```

A partir d'aquesta descripció i definicions inicials, es demana implementar les següents funcions:

- a. **(0.5 punts)** void inicialitza_sensors (t_sistema *sistema); aquesta funció ha d'inicialitzar el sistema apuntat pel paràmetre sistema, indicant que, de moment, cap dels seus NUM_S sensors ha pres encara cap mesura.

```

void inicializa_sensors (t_sistema *sistema)
{
    int i;

    for (i=0; i<NUM_S; i++)
        sistema->sensors[i] = NULL;
}

```

- b. **(1 punt)** int afegir_mesura (t_sistema *sistema, int pos, int hora, float valor); aquesta funció ha d'afegir una nova mesura del sensor representat per la posició pos de l'array de sensors del sistema apuntat pel paràmetre sistema. En cas que la posició pos proporcionada com a paràmetre sigui errònia, la funció ha d'acabar retornant -1. En cas d'ocórrer qualsevol problema durant la reserva de la memòria dinàmica necessària per a la nova mesura, la funció ha d'acabar retornant el valor -2. Altrament, la nova mesura, correctament incialitzada amb la hora i valor proporcionats com a paràmetre, cal afegir-la al final de la llista de mesures del sensor. Si la operació pot realitzar-se de forma correcta, la funció cal que acabi retornant 0.

```

int afegir_mesura (t_sistema *sistema, int pos, int hora, float valor)
{
    t_node *nou, *tmp;

    if (pos < 0 || pos >= NUM_S)
        return (-1);

    nou = (t_node *)malloc (sizeof (t_node));
    if (nou == NULL)
        return (-2);

    nou->hora = hora;
    nou->valor = valor;
    nou->next = NULL;

    if (sistema->sensors[pos] == NULL)
        sistema->sensors[pos] = nou;
    else
    {
        tmp = sistema->sensors[pos];
        while (tmp->next != NULL)
            tmp = tmp->next;

        tmp->next = nou;
    }

    return (0);
}

```

- c. **(2 punts)** float * extreure_mesures (t_sistema *sistema); aquesta funció ha de retornar un punter a un nou array dinàmic de variables de tipus float, que contingui a les seves posicions els valors de totes les mesures preses pels NUM_S sensors del sistema apuntat pel paràmetre sistema. Abans de crear-lo, caldrà per tant conèixer la suma total de mesures preses per tots els sensors del sistema IoT, doncs aquest valor en determina la seva mida. En cas d'ocórrer qualsevol problema durant la reserva de la memòria dinàmica necessària per al nou array dinàmic, la funció ha d'acabar retornant NULL.

```

float * extreure_mesures (t_sistema *sistema)
{
    float *valors;
    t_node *tmp;
    int comptador = 0, i, j;

    for (i = 0; i < NUM_S; i++)
    {
        if (sistema->sensors[i] != NULL)
        {
            tmp = sistema->sensors[i];
            while (tmp != NULL)
            {
                comptador++;
                tmp = tmp->next;
            }
        }
    }

    return valors;
}

```

```

    valors = (float *)calloc (contador, sizeof(float));
    if (valors == NULL)
        return (NULL);

    j = 0;
    while (j < comptador)
    {
        for (i = 0; i < NUM_S; i++)
        {
            if (sistema->sensors[i] != NULL)
            {
                tmp = sistema->sensors[i];
                while (tmp != NULL)
                {
                    valors[j++] = tmp->valor;
                    tmp = tmp->next;
                }
            }
        }
        j++;
    }

    return (valors);
}

```

- d. **(1 punt)** void mostrar_mesures (t_sistema *sistema, int horaIni, int horaFin); aquesta funció ha de mostrar totes les mesures dels sensors del sistema apuntat pel paràmetre sistema preses entre les hores [horaIni, horaFin]. En cas que horaFin sigui erròniament anterior a horaIni, la funció ha de finalitzar directament, sense mostrar cap informació per pantalla. A continuació es mostra un exemple d'execució esperada de la funció, quan les hores proporcionades com a paràmetre són correctes:

Mesures entre 10:45h i 11:00h	Sensor 6:
Sensor 1:	Sensor 7:
10:47 3.40	Sensor 8:
10:57 5.20	Sensor 9:
Sensor 2:	10:50 24.22
10:45 1.12	Sensor 10:
Sensor 3:	10:45 12.11
Sensor 4:	10:50 11.55
Sensor 5:	10:55 10:33
	11:00 12:12

```

void mostrar_mesures(t_sistema *sistema, int horaIni, int horaFin)
{
    int i;
    t_node *tmp;

    if (horaIni > horaFin)
        return;

    printf("Mesures entre %02d:%02dh i %02d:%02dh\n\n", horaIni/100,
        horaIni%100, horaFin/100, horaFin%100);
}

```

```

for (i = 0; i < NUM_S; i++)
{
    printf("Sensor %d:\n", i+1);

    if (sistema->sensors[i] != NULL)
    {
        tmp = sistema->sensors[i];
        while (tmp != NULL)
        {
            if (tmp->hora >= horaIni && tmp->hora <= horaFin)
            {
                printf ("%02d:%02d %.02f\n",
                    tmp->hora/100, tmp->hora%100, tmp->valor);
            }

            tmp = tmp->next;
        }
    }
}

```

PROBLEMA 3: 1.5 punts, temps de resolució: 25 min.

Implementa la funció:

```
int substituir_bits(unsigned int *seq_d, unsigned int seq, int b1, int b2);
```

Aquesta funció rep com a paràmetre un punter (seq_d) a variable entera sense signe (unsigned int), així com una altra variable entera sense signe (seq), que guarden seqüències binàries de 32 bits. També rep dues posicions de bit (b1 i b2), el valor de les quals ha d'estar comprès entre [0, 31] i b1 >= b2, doncs b1 representa una posició igual o més significativa que b2 en les seqüències binàries amb les que treballem.

En cas que el valor del paràmetre b1 i/o b2 sigui incorrecte, la funció ha d'acabar retornant directament -1.

Altrament, la funció ha de substituir els bits ubicats entre les posicions b1 i b2 (ambdues incloses) de la seqüència binària apuntada pel paràmetre seq_d, de forma que prenguin els mateixos valors que els bits de la seqüència binària guardada pel paràmetre seq, en aquelles mateixes posicions. A més, ha d'acabar retornant el nombre de bits entre [b1, b2] de la seqüència apuntada per seq_d que han canviat el seu valor durant l'operació.

Per exemple, assumim que les seqüències de bits involucrades són inicialment igual a:

```

seq_d -> 1010 1010 1010 1010 1010 1010 1010 1010 (0xAAAAAAAA)
seq      : 0001 0001 0001 0001 0001 0001 0001 0001 (0x11111111)

```

En passar-li b1 = 13 i b2 = 4 com a paràmetre a la funció, el valor que retornaria aquesta seria 8 (canvia el valor de 8 bits en total de la seqüència apuntada pel paràmetre seq_d), obtenint una seqüència resultant de l'operació igual a:

```
seq_d -> 1010 1010 1010 1010 1001 0001 0001 1010 (0xAAAA911A)
```

Nota: cal aconseguir la funcionalitat desitjada utilitzant exclusivament operadors a nivell de bit. No s'acceptaran respostes que utilitzin operadors aritmètics per tal d'aconseguir el mateix propòsit.

```
int substituir_bits(unsigned int *seq_d, unsigned int seq, int b1, int b2)
{
    int i, canvis_bit = 0;
    unsigned int mask;

    if (b1 < 0 || b1 > 31 || b2 < 0 || b2 > 31 || b1 < b2)
        return (-1);

    for (i = b1; i >= b2; i--)
    {
        mask = 1 << i;

        if ((*seq_dest & mask) != (seq & mask))
        {
            canvis_bit++;
            *seq_dest = *seq_dest ^ mask;
        }
    }

    return (canvis_bit);
}
```